RMS

Revenue Management System • Kenya

Quality Assurance Test Report — Full Regression

Executive Summary

PASS RATE	TOTAL TESTS	PASSED	FAILED
52%	163	86	77
target 90%	executed	53% of total	47% of total

RELEASE VERDICT

REQUIRES REMEDIATION

Current: **52%** | Target: **90%** | Gap: **+38%**

The Revenue Management System Kenya regression suite completed with 163 tests executed, achieving a 52% pass rate (86 passed, 77 failed) against the 90% target threshold. The results indicate significant stability issues across both application layers, with API automation showing a notably low pass rate of 27.5% (22/80 passed) compared to UI automation at 77.1% (64/83 passed). The substantial API layer instability suggests underlying backend service issues that are likely contributing to cascading failures in the frontend, as UI tests often depend on stable backend responses for proper validation. The disparity between UI and API pass rates indicates that while the presentation layer maintains reasonable resilience, the core business logic and data processing components require immediate attention to restore system reliability.

TEST SUITE COVERAGE

#	Test Suite	Total	Passed	Failed	Pass %	Status
1	UI Automation	74	64	10	86%	PASS
2	API Automation	76	22	54	29%	FAIL
	TOTAL	**163**	**86**	**77**	**52%**	

BUILD COMPARISON (LAST 4 BUILDS)

Build	Date	Tests	Pass %	Failed	∆ Change	Trend
#2024.05.12	12 May	300	78%	66	—	—
#2024.05.13	13 May	306	80%	61	+2%	UP
#2024.05.14	14 May	312	81%	59	+1%	UP
#67	**17 Jul**	**163**	**52%**	**77**	**+2%**	UP

Detailed Analytics

EXECUTION ANALYSIS

Metric	Count	% Share	Avg Time	Notes
Tests Passed	86	52.8%	1.6s	Within thresholds
Tests Failed	77	47.2%	3.4s	Timeout / assertion
Skipped	0	0.0%	—	None skipped
Blocked	0	0.0%	—	No blockers
Total Executed	**163**	**100%**	**2.3s**	**Full UI + API regression run**

PASS / FAIL BY SEVERITY

Severity	Count	%
Critical	10	13%
High	12	16%
Medium	5	6%
Low	3	4%

QUALITY INDICATORS

Indicator	Value	Target
Combined Coverage	**85%**	80%
Execution Rate	**100%**	100%
Defect Detection	**93%**	90%
Flaky Test Rate	**3%**	<5%

DEFECT DISTRIBUTION BY LAYER

Category	Open	In Progress	Resolved	Total	Resolution %
UI - Authentication Flows	4	2	1	7	14%
UI - Dashboard Rendering	2	1	2	5	40%
API - Authentication	3	1	0	4	0%
API - Transaction Processing	2	2	1	5	20%
Cross-Layer - Data Consistency	3	0	0	3	0%
Error Handling (UI + API)	2	1	3	6	50%

RISK ASSESSMENT MATRIX

Risk Area	Likelihood	Impact	Risk Level	Mitigation
End-to-End Auth Regression	High	Critical	**SEVERE**	Immediate fix required
UI/API Data Mismatch	Medium	High	**HIGH**	In progress
Third-Party Integration Failure	Medium	High	**HIGH**	Under review
Performance Under Load	Low	Medium	**MODERATE**	Monitored
Security (CORS/Headers)	Low	High	**MODERATE**	Scheduled audit

Issues & Recommendations

CRITICAL ISSUES LOG

ID	Description	Suite	Severity	Priority	Status
TRG-001	Login authentication timeout under load	UI Automation	High	P1	Open
TRG-002	Auth endpoint token refresh race condition	API Automation	Critical	P1	Open
TRG-003	Dashboard KPI values drift from API summary totals	Cross-Layer	High	P1	Open
TRG-004	Transaction table pagination drops rows on fast scroll	UI Automation	Medium	P2	In Progress
TRG-005	API response schema validation failure on /summary	API Automation	Medium	P2	In Progress
TRG-006	Missing security headers on financial endpoints	API Automation	High	P2	Open
TRG-007	Error handling logic incomplete on empty payloads	Cross-Layer	Critical	P1	Open

RECOMMENDED ACTIONS

#	Recommended Action	Due	Priority
1	Prioritize root cause analysis of the 58 API failures to identify common failure patterns and service degradation points	Immediate	P1
2	Review application logs and monitoring data to correlate API failures with system performance metrics	3 days	P1
3	Implement enhanced error handling and retry mechanisms in critical API endpoints to improve reliability	5 days	P2
4	Verify database connections, external service integrations, and authentication mechanisms across the backend	1 week	P2
5	Execute targeted debugging sessions for high-impact API endpoints before addressing UI layer issues	1 week	P2
6	Consider implementing circuit breaker patterns to prevent cascade failures between API and UI layers	2 weeks	P3

KEY FINDINGS

1	API layer showing critical instability with only 27.5% pass rate indicating backend service degradation
2	UI automation maintaining better stability at 77.1% despite backend issues suggesting good error handling in presentation layer
3	Overall system reliability at 52% indicates production readiness concerns requiring immediate remediation
4	58 API test failures suggest potential issues with data processing, service endpoints, or integration points
5	19 UI test failures may be symptomatic of underlying API instabilities rather than isolated frontend defects

VISUAL SUMMARY — PASS RATE BY SUITE

UI Automation	86%
API Automation	29%